CS 334 : COMPILER DESIGN LABORATORY **Scheduled : Feb 17 to 20, 2026**
Premises : Lab 5 [VIA], Lab 1 [VIB] and Lab 7[VIC]

Faculty : Bikas Kanti Sarkar, Indrajit Mukherjee, Prashant Pranab, Anup Keshri, Supratim Biswas
Teaching Assistant : Swarna Aishwarya Twinkle (Research Scholar)

Lab Assignment 5

Objective : 1. Improve communication between scanner and parser by learning new features.
2. Understand the files produced by yacc, "y.tab.h", "y.output" and "y.tab.c" and their role in working with an LALR(1) parser.
3. Work with unambiguous expression grammar, generate a parser and test its working.
4. Work with an ambiguous expression grammar, generate a parser and test its working. Add disambiguating rules of yacc to an ambiguous grammar, generate a parser and test its working.
5. Generate a parser with debugging option enabled, test the working of the generated parser and interpret the results of the parser.

General Instructions :

- Generation of scanner and parser
  $ lex file.l $ yacc -dv file.y $ gcc lex.yy.c y.tab.c -o parser
- Test the parser on sample input string, such as “input.txt” and save the generated output.
  $ ./ parser < input.txt > output.txt

P1. The objective is to design a parser for simple expressions. We start with 2 operators, ‘+’ and ‘%’ and unsigned integer as operands. The rules of the grammar are {E → E + T | T; T → T % F | F; F → NUM}.
(a) Read the 2 files given to you, “expr5.1.l” and “expr5.1.y” and make your observations. Generate an expression parser using the instructions given above. Read the handout to understand the communication between the scanner and parser in this set up.

(b) Save the file, "y.output" as "y1.output". Construct LALR(1) parsing table manually from the information given in this file. Read the handout for help.

(c) Run the generated parser over sample input files, "inp1.txt" and "inp2.txt", explain the behavior of the parser for both the outputs produced.

(d) It is possible to trace the working of a parser generated by yacc, by performing the following actions.

- Remove the comment in the main() in yacc script as follows.
  Change // yydebug = 1; to yydebug = 1;
- Use the debugging switch "-t" when running yacc, such as $ yacc -dv -t expr5.1.y
- Generate parser as usual and run the parser on an input file.
  $ gcc lex.yy.c y.tab.c -o expr1new $ ./expr1new < inp1.txt

Reason the output generated by the above command using the y.output file for this run.

P2. In P1, the properties of operators, both precedence and associativity, are hard coded in the unambiguous grammar used. Write an unambiguous grammar so that the properties of the operators are as given below. Higher precedence value of an operator denotes higher priority.

Precedence	Associativity	Arity	Operator
13	R	binary	+
12	L	binary	%

Generate a parser with the new grammar, say expr2; run the input file, "inp1.txt" with this parser and explain its behavior.

P3. Change the grammar of P1 to an ambiguous grammar as given below.

E → E % E | E + E | NUM

Write the grammar in yacc notation, add actions, similar to those given in "expr5.1.y" and generate a new parser, say, expr3.

Write down all your observations, in the process of generating the parser, and also the working of this parser on an input file, "inp1.txt" or "inp2.txt". Give appropriate reasons for the observations, wherever possible.

P4. Yacc has features to use an ambiguous grammar but use its rules for disambiguation to generate a parser that does the correct actions as if the grammar was unambiguous. These features are described below. Besides the "%token" feature to define the terminals of interest, "%left" and "%right" are used to set the associativity of operators. Operators having the same precedence and same associativity can be defined together. The operators are listed in the order from least to highest precedence value. For example,

```
%left '+'  '-'
%right  '*'  '/'  '%'
```

the above specifications informs yacc the following information :

1. Both operators '+' and '-' have the same precedence and are left associative
2. The operators '*' '/' and '%' have same precedence value and all are right associative; since these are listed below '+' and '-' , these 3 operators have higher precedence than '+' and '-'

Use the ambiguous grammar of P3 and use the disambiguation rules of yacc to generate a parser that is equivalent to the parser of P1. Execute your parser on sample input file and explain the behavior of the parser.

P5. Consider the expression given below which uses the subset of C operators (given in Appendix) and non-negative integers.

```
1 <<  4 / 2 + 1 == 2 * 4 && 2 > 0 || 5 < 6
```

(a) Manually construct a parse tree for the above expression that satisfies all the properties of the operators and find the value of the expression at each node of the parse tree and finally at the root. Write a C program and check that the value manually calculated by you matches with the output of the program.

(b) Write a minimal lex script and an ambiguous grammar that can generate expressions using the operators given in the Appendix. Use the disambiguating rules of yacc to generate a conflict free parser. Test the working of your parser with the input expression given above. If time permits, test your parser with sample inputs constructed by you.

End of the Experiment 5

APPENDIX : OPERATORS IN C

Precedence	Associativity	Arity	Operator	Function
13	L	binary	*, /, %	multiplicative operators
12	L	binary	+, -	arithmetic operators
11	L	binary	<<, >>	bitwise shift
10	L	binary	<, <=	relational operators
9	L	binary	==, !=	equality, inequality
5	L	binary	&&	logical AND
4	L	binary	||	logical OR